

CPSC 426 Assignment 4: Cloth Simulation

Due 11:59PM, March 25, 2026

1 Introduction

In this assignment you will implement a GPU-accelerated cloth simulation using the *Fast Mass-Spring* method of Liu et al. [1]. Starting from a mass-spring model of a cloth sheet, you will build:

1. **Implicit Euler via Energy Minimization:** formulate the time-stepping as an optimization problem and implement the energy terms to be minimized.
2. **Block Coordinate Descent (BCD) Solver:** implement the local-global alternating optimization loop with gradient-based position updates.
3. **Collision Response:** detect and resolve collisions between the cloth and rigid-body colliders using BVH-accelerated closest-point queries.

All GPU kernels are written with NVIDIA Warp [2], and outputs are exported as USD files viewable in Blender. You will work from the provided starter code; your task is to implement the missing kernel and function logic. The starter code provides `TODO` comments as hints, but you are expected to consult the reference paper [1] (included as `liu13fast.pdf`) and the lecture notes for the mathematical formulations:

- **Lecture 14:** Optimization-based integrators, energy formulation (Section 3 of [1]).
- **Lecture 15:** Block coordinate descent, local/global decomposition (Section 3.1 of [1]).
- **Lecture 16:** Solving the global step, preconditioning (Section 3.2 of [1]).

The assignment is designed so that each part can be verified visually before moving on:

- After **Part I**, a pinned cloth falls and drapes under gravity (no colliders).
- After **Part II**, the cloth interacts with rigid-body colliders (spheres, planes).

1.1 Getting the Code

Assignment code is hosted on the course's GitHub repository:

<https://github.students.cs.ubc.ca/CPSC426-2025W-T2/a4-release>

1.2 Template

1.2.1 Source Files

- `main.py`: CLI entry point, simulation loop, and USD rendering. You do *not* need to modify this file.
- `config.py`: `SimConfig` dataclass containing all simulation parameters (timestep, stiffness, solver iterations, collision margin, etc.). You do *not* need to modify this file.
- `mesh.py`: Cloth grid generation, collider OBJ loading, and procedural mesh generators. You do *not* need to modify this file.
- `solver/energy.py`: Energy evaluation kernels.
You will implement `compute_inertia_target`, `eval_inertia_energy`, and `eval_spring_energy`.
- `solver/local_step.py`: Local step kernel.
You will implement `local_step_kernel`.
- `solver/global_step.py`: Global step kernel.
You will implement `gradient_step_kernel`.
- `solver/integrator.py`: `ClothSimulation` class.
The constructor (`__init__`) provides all array allocations except `A_diag`, which you will compute (Task 4a). Helper methods (timing, convergence reporting) are provided. You will implement the `step()` method, which orchestrates the full BCD loop.
- `collision/response.py`: Collision detection and response.
The `DynamicCollider` class, `build_collider`, and `merge_collider_meshes` are provided. You will implement `collision_kernel`.
- `examples/`: Example scripts for testing. You do *not* need to modify these, but you should use them to validate your implementation.

1.2.2 Data and Assets

The `assets/` directory stores procedurally generated collider meshes (sphere, plane OBJ files). These are created automatically by the example scripts if not already present. The reference paper `liu13fast.pdf` is included in the root directory.

1.2.3 Dependencies

```
conda create -n cpssc426 -c conda-forge --override-channels \
    python=3.14.2 numpy pip openusd -y
conda activate cpssc426
pip install warp-lang trimesh
```

1.3 Execution

```
# Part I: cloth falling under gravity (no collision)
python examples/falling_cloth.py

# Part II: cloth draping onto a sphere
python examples/cloth_on_sphere.py

# Part II: dynamic collider pushing pinned cloth
python examples/cloth_pushed_by_sphere.py

# Part II: multiple colliders (sphere + plane)
python examples/cloth_on_sphere_and_plane.py

# General CLI
python main.py --res 32 --duration 5.0 --out cloth.usd
python main.py --res 32 --precond --collider assets/sphere.obj
```

Output .usd files can be viewed in Blender (File → Import → Universal Scene Description).

Please also read the README.md for additional setup instructions.

2 Algorithm Overview

This section summarizes the simulation pipeline. The algorithm was covered in Lectures 14–16; refer to those notes and Sections 2–3 of [1] for derivations.

Core idea. Implicit Euler time-stepping is cast as an energy minimization (Lecture 14, Section 3 of [1]):

$$\mathbf{x}^{n+1} = \arg \min_{\mathbf{x}} E(\mathbf{x})$$

where E is the sum of an inertia term and a spring potential. This is solved via block coordinate descent (Lecture 15): auxiliary spring-direction variables $\mathbf{d}$ are introduced, and the solver alternates between a closed-form *local step* (optimize $\mathbf{d}$) and a gradient-based *global step* (optimize $\mathbf{x}$). The global step uses either plain gradient descent or Jacobi-preconditioned gradient descent (Lecture 16).

Per-timestep pipeline (what `step()` implements):

1. Save $\mathbf{x}_{\text{old}} \leftarrow \mathbf{x}$; compute inertia target $\mathbf{y}$; initialize $\mathbf{x} \leftarrow \mathbf{y}$.
2. Update dynamic collider position (if present).
3. **BCD loop** (up to `num_bcd_iters`):
 - (a) *Local step*: update $\mathbf{d}$ from current $\mathbf{x}$ (closed-form).
 - (b) *Global step*: gradient descent on $E(\mathbf{x})$ (up to `num_gd_iters`, using `wp.Tape` for autodiff).
 - (c) *Collision*: project penetrating vertices outside collider surface.
 - (d) Check convergence; break early if converged.
4. Update velocities: $\mathbf{v} \leftarrow (\mathbf{x} - \mathbf{x}_{\text{old}})/h$.

3 Work to be Done (100 pts)

3.1 Part I: Mass-Spring Simulation (55 pts)

In this part you implement the core simulation pipeline: energy formulation, BCD solver, and time integration. The result is a cloth that falls and drapes under gravity with pinned vertices, without collision handling.

Task 1 (5 pts): Inertia Target (`energy.py: compute_inertia_target`).

Compute the inertia target $\mathbf{y}$ for each vertex: the position it would reach under gravity if there were no springs. Pinned vertices remain fixed. See the optimization formulation in Lecture 14 / Section 3 of [1].

Task 2 (15 pts): Energy Evaluation (`energy.py: eval_inertia_energy` and `eval_spring_energy`).

Implement the two energy terms that define the objective $E(\mathbf{x})$ minimized by the BCD solver. The inertia energy is a per-vertex mass-weighted term; the spring energy is a per-spring stiffness-weighted term involving the auxiliary direction $\mathbf{d}_s$. Derive both from Lecture 14 / Section 3 of [1]. Accumulate each contribution into the shared `loss` array via `wp.atomic_add`. These kernels must be differentiable (they run inside `wp.Tape`).

Task 3 (5 pts): Local Step (`local_step.py: local_step_kernel`).

For each spring, update $\mathbf{d}_s$ given the current positions. This is the closed-form solution from Lecture 15 / Section 3.1 of [1]. Note that $\mathbf{d}_s$ encodes both the direction and the rest length of the spring.

Task 4 (30 pts): Global Step and BCD Loop (`global_step.py: gradient_step_kernel`; `integrator.py: __init__` and `step()`).

This task has three parts. **We strongly recommend implementing plain gradient descent first (part b), then the BCD loop (part c), and adding Jacobi preconditioning (part a) last.** The `falling_cloth.py` example works with plain GD; the collision examples use Jacobi.

(a) **Jacobi diagonal precomputation** (`integrator.py: __init__`).

Compute `self.A_diag`, a per-vertex diagonal preconditioner. For each vertex i :

$$A_{ii} = m_i + h^2 \sum_{s: i \in s} k_s$$

where m_i is the vertex mass, h is the timestep, and k_s is the stiffness of spring s . The sum runs over all springs that connect to vertex i .

Why this diagonal? The total energy $E(\mathbf{x})$ is the sum of inertia and spring terms. Its Hessian $\nabla^2 E$ has a block-diagonal structure where each 3×3 block corresponds to a vertex. The diagonal entry A_{ii} captures the mass contribution from the inertia term and the stiffness contributions from all springs attached to vertex i , scaled by h^2 (see Lecture 16 / Section 3.2 of [1]). Using $1/A_{ii}$ as a per-vertex step size is equivalent to a Jacobi iteration on the linear system, which converges much faster than plain GD with a fixed step size, especially for stiff systems.

This diagonal is constant (it depends only on topology, masses, and stiffnesses) and is computed once in the constructor. The stub in `__init__` indicates where to place this computation. All other array allocations are provided.

(b) **Gradient step kernel** (`global_step.py`).

Implement the per-vertex position update. The kernel must support *two separate modes*, selected by the `use_precond` flag:

- **Mode 1: Plain gradient descent** (`use_precond=0`): $\mathbf{x}_i \leftarrow \mathbf{x}_i - \alpha \nabla_{\mathbf{x}_i} E$
Uses a fixed step size α (from `SimConfig.gd.alpha`). Simple but requires careful tuning of α and converges slowly for stiff problems.
- **Mode 2: Jacobi preconditioning** (`use_precond=1`): $\mathbf{x}_i \leftarrow \mathbf{x}_i - \nabla_{\mathbf{x}_i} E / A_{ii}$
Replaces the fixed step size with the inverse Hessian diagonal. No step-size tuning is needed; converges significantly faster. The α parameter is ignored in this mode.

Pinned vertices must not be updated in either mode.

(c) **BCD loop** (`integrator.py: step()`).

Assemble the complete per-timestep pipeline from Section 2. Use `wp.Tape` to record the energy evaluation (Tasks 1–2), call `tape.backward()` for gradients, launch the gradient step kernel, then reset gradients and tape. Use the provided `_check_gd_convergence(prev_energy)` and `_check_bcd_convergence(prev_energy)` helper methods for early termination of the inner GD loop and outer BCD loop respectively. Call `_begin_step()` at the start and `_end_step()` at the end of your implementation. Use `self._timer("name")` to wrap each phase for timing.

Do *not* include collision handling yet; leave a placeholder for Part II.

The constructor provides all Warp array allocations (`self.x`, `self.x_old`, `self.v`, `self.y`, `self.d`, `self.loss`, `self.mass`, `self.is_pinned`, `self.spring_indices`, `self.rest_lengths`, `self.stiffness`) and timing/convergence infrastructure. You must compute `self.A_diag` yourself. The following helper methods are also provided:

- `_timer(name)`: context manager for timing phases (e.g., with `self._timer("local_step")`).
- `_begin_step()` / `_end_step()`: call at the start/end of `step()` for per-frame bookkeeping.
- `_check_gd_convergence(prev_energy)`: returns (`converged`, `energy`) after each GD iteration.
- `_check_bcd_convergence(prev_energy)`: returns (`converged`, `energy`) after each BCD iteration; internally tracks energy history.
- `_eval_total_energy(h)`, `print_timing_summary()`, `print_convergence_summary()`: diagnostics.

Checkpoint:

After completing Tasks 1–4, run `examples/falling_cloth.py`. A cloth sheet pinned at two top corners should fall and drape under gravity. Open the output `cloth_falling.usd` in Blender to verify. If the cloth explodes or does not move, check your energy formulations and gradient step carefully.

3.2 Part II: Collision Response (30 pts)

In this part you will implement collision detection and response, enabling the cloth to interact with rigid-body colliders.

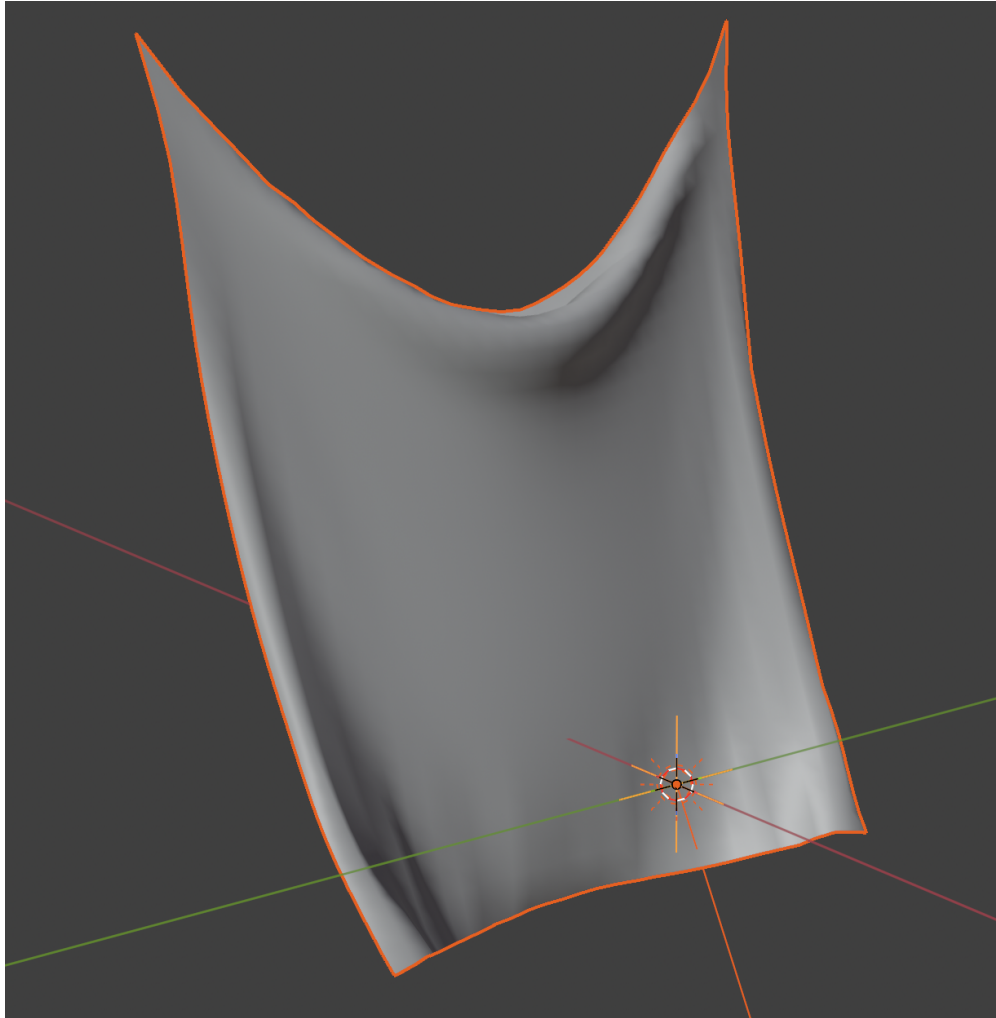


Figure 1: Expected result for `falling_cloth.py`: a cloth sheet pinned at two top corners draping under gravity.

Task 5 (15 pts): Collision Kernel (`collision/response.py`: `collision_kernel`).

Implement the collision response kernel. For each cloth vertex, this kernel detects penetration into a collider mesh and projects the vertex to the collider surface.

The algorithm for each vertex:

1. Query the closest point on the collider mesh using `wp.mesh_query_point`.
2. If the query succeeds, reconstruct the closest surface point using `wp.mesh_eval_position`.
3. Determine if the vertex is *penetrating* (inside the collider) or *within the collision margin*.
4. If so, compute the outward surface normal from the vertex-to-closest-point direction (use the sign from the query to ensure correct orientation). For vertices exactly on the surface, fall back to the face normal via `wp.mesh_eval_face_normal`.
5. Project the vertex to: $\mathbf{x}_i \leftarrow \mathbf{p}_{\text{closest}} + \hat{\mathbf{n}} \cdot \text{margin}$.

Pinned vertices should be skipped. Consult the Warp documentation for the `wp.Mesh` query API: `wp.mesh_query_point`, `wp.mesh_eval_position`, and `wp.mesh_eval_face_normal`.

Task 6 (15 pts): Integrating Collision into the Solver.

Extend your `step()` method from Task 4 to include collision response. Specifically:

- Update the dynamic collider's position before the BCD loop begins (the `DynamicCollider.update(t)` method is provided).
- After the global step phase (the inner GD loop) completes within each BCD iteration, launch the collision kernel to project penetrating vertices.
- Verify that collision works with both static and dynamic colliders, and with multiple merged colliders.

The collision kernel should be called with the collider's mesh ID (`self.collider.mesh.id`), the collision margin from `SimConfig`, and a maximum query distance (e.g., 2.0 m).

Checkpoint:

After completing Tasks 5–6, run the collision examples:

- `examples/cloth_on_sphere.py`: a free cloth drapes over a sphere.
- `examples/cloth_pushed_by_sphere.py`: a sphere descends onto a pinned cloth.
- `examples/cloth_on_sphere_and_plane.py`: cloth falls onto a sphere and a ground plane (tests multi-collider support).

The cloth should rest on or wrap around the collider surfaces without penetrating them.

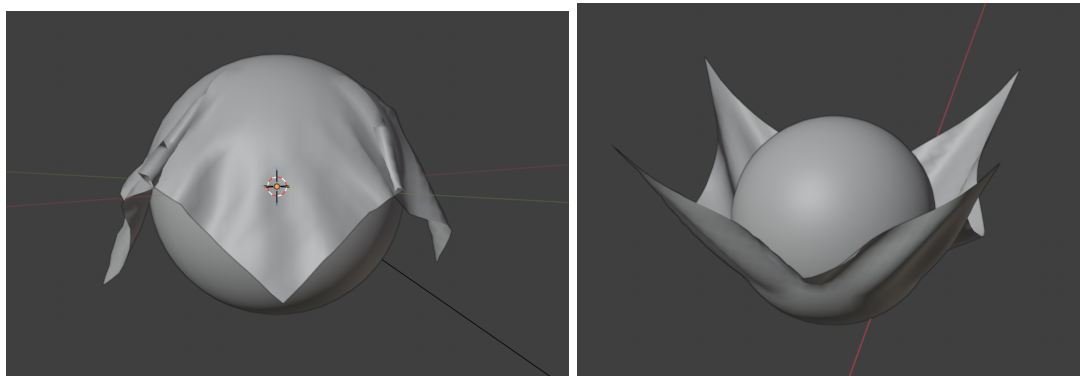


Figure 2: Expected results for collision examples. Left: `cloth_on_sphere.py` — a free cloth drapes over a sphere. Right: `cloth_pushed_by_sphere.py` — a sphere descends onto a pinned cloth.

3.3 Part III: Feature Extension (15 pts)

Extend the assignment with a feature of your choice. You have full creative freedom here. Consider what aspects of the simulation could be improved, what physical phenomena are missing, or what

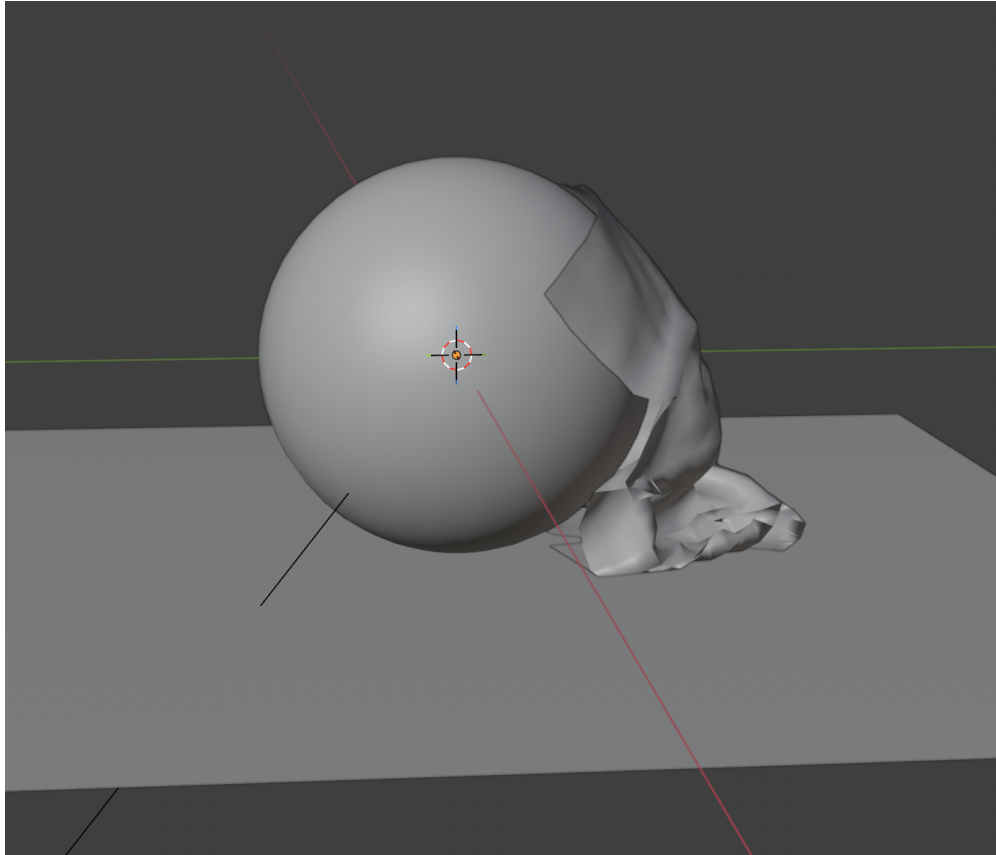


Figure 3: Expected result for `cloth_on_sphere_and_plane.py`: cloth falling onto a sphere and a ground plane.

interesting scenarios could be demonstrated. The best extensions show both technical depth and thoughtful motivation. Think about what you found most interesting in the lectures or paper and explore it further.

Here are some suggestions to get you started (you are not limited to these):

- **Wind / aerodynamic forces:** Apply per-triangle wind forces based on face normals and a global or spatially varying wind field.
- **Self-collision:** Detect and resolve collisions between different parts of the cloth itself using spatial hashing or BVH queries.
- **Friction:** Add tangential friction to the collision response so the cloth grips surfaces realistically instead of sliding freely.
- **Tearing / cutting:** Allow springs to break when stretched beyond a threshold, simulating cloth tearing.
- **Better convergence:** Implement L-BFGS or Chebyshev semi-iterative acceleration for the global step and compare convergence rates against plain GD and Jacobi.

4 Summary of Tasks

#	File	Kernel / Function	Part	Pts
1	<code>solver/energy.py</code>	<code>compute_inertia_target</code>	I (Energy)	5
2	<code>solver/energy.py</code>	<code>eval_inertia_energy</code> + <code>eval_spring_energy</code>	I (Energy)	15
3	<code>solver/local_step.py</code>	<code>local_step_kernel</code>	I (Local)	5
4	<code>solver/global_step.py</code> <code>solver/integrator.py</code>	<code>gradient_step_kernel</code> + <code>A_diag</code> in <code>__init__()</code> + <code>step()</code>	I (Global/BCD)	30
5	<code>collision/response.py</code>	<code>collision_kernel</code>	II (Collision)	15
6	<code>solver/integrator.py</code>	collision integration in <code>step()</code>	II (Collision)	15
7	Feature Extension		III	15
			Total	100

Recommended order: Tasks 1 → 2 → 3 → 4 (verify with `falling_cloth.py`) → 5 → 6 (verify with collision examples).

5 Tips

- Review Lectures 14–16 and Sections 2–3 of [1] before starting.
- Use `wp.atomic_add` to accumulate per-element energy contributions into the shared scalar `loss`. Zero the loss (`self.loss.zero_()`) before each evaluation; zero the gradient (`self.x.grad.zero_()`) and call `tape.reset()` after each gradient step.
- If the cloth explodes, common causes include incorrect energy scaling (missing h^2 or $\frac{1}{2}$), wrong sign in the gradient step, or forgetting to reset tape/gradients between iterations.
- The collision kernel uses Warp’s mesh query API. Consult the Warp documentation for `wp.mesh_query_point`.

6 Submission Instructions

6.1 Directory Structure

Your submission should maintain the provided directory structure so that in F2F grading we can easily run the example scripts. For feature extensions, please create a separate directory (e.g., `extensions/`) including your changes. If your feature extension requires more complex separation, you can tweak this. We only ask that you do not duplicate excessively.

Please add your name, student number, and CWL username in the existing `README.md` file. You should modify the first line’s template and replace it with your information. Also, you can include any information you would like to pass on to the marker below that (otherwise leave N/A is fine).

6.2 Submission Methods

Please compress everything under the root directory of your assignment into an **a4.zip** and submit it on Canvas. You can make multiple submissions, but we will grade only the last one.

7 Grading

7.1 Face-to-face (F2F) Grading

To get a grade for the assignment, you must meet face-to-face with a TA in a ~10-min slot on Zoom to demonstrate that you understand how your program works. Grading slots and detailed instructions on how to sign up and conduct F2F grading will be announced on Canvas and on Piazza.

While the exact process and questions may vary, you can generally expect the following during the meeting: The TA will (1) look at and/or run your code to inspect its correctness; (2) ask you to explain parts of your code, **including Python syntax and library features you use**; (3) ask you understanding-based questions about the assignment and logic. You are expected to answer them in a reasonable timeframe (based on difficulty).

The questions are intended to be mostly based on the code we ask you to implement, so implicit Euler integration, energy minimization, block coordinate descent, spring projections, gradient descent with preconditioning, collision detection/response, and Warp concepts that you will have seen in lecture and while completing the assignment. We hope that the questions are more conversational than robotic; not everything we say is a test to deduct points from you.

In most cases, any awkwardness or nervousness is just noise, so feel free to take pauses and ask for clarifications if anything confuses you. We may ask questions that probe deeper into “why,” and from that perspective it is okay to not immediately have an answer. We want to see your thought process in figuring out one.

7.2 Point Allocation

All questions before Feature Extension have a total of 85 points. The points are warranted based on

- The functional correctness of your program, i.e., how visually close your results are to expected results;
- The algorithmic correctness of your program, e.g., correctly implementing the energy kernels, BCD loop, collision response, and preconditioning;
- Your demonstration of understanding of the code through your answers to TAs’ questions during face-to-face (F2F) grading.

The feature extension grade is based on creativity and sophistication of your submission. We warrant partial marks on four scales:

- **15:** Outstandingly creative and sophisticated feature extension. Note that only a very small number of extensions may get this score.
- **10:** Average creativity and sophistication.
- **5:** Basic creativity and sophistication, e.g. uses the existing APIs to combine into a novel result.
- **0:** Trivial or no feature extension, e.g. only changes parameters, makes purely cosmetic tweaks without new logic, or adds an extension that does not work reliably or cannot be explained during grading.

7.3 Collaboration Policy

Submissions must be individual, but you are encouraged to discuss the assignment with your classmates, and you are allowed to use generative AI tools, e.g. ChatGPT, Claude to help with constructing your solution.

In the beginning lines of the README.md file, include names of the people you discussed the assignment with, websites you acquired information from, and if you used generative AI tools, include the names of the tools you used.

You must understand your code and be able to explain it during F2F grading. If you cannot demonstrate at least a basic understanding of your code during face-to-face grading, you may receive a zero for the assignment, even if your code runs perfectly. Hiring a tutor to write your assignment, or copy-pasting others' code without proper referencing in README.md are considered as plagiarism, and will be reported to the dean's office.

7.4 Penalties

Aside from penalties from incorrect solution or plagiarism, we may apply the following penalties to each assignment:

Late penalty. You are entitled up to three grace (calendar) days in total throughout the term. No penalties would be applied for using them. However, once you have used up the grace days, a deduction of 10 points would be applied to each extra late day. Note that

1. The three grace days are given for all assignments, **not per assignment**, so please use them wisely;
2. We check the time of your last submission to determine if you are late or not.

No-show penalty. Please sign up for a grading slot on the provided sign-up spreadsheet (link will be posted later on Piazza) before the submission deadline of the assignment, and show up to your slot on time. A 10-point deduction would be applied to each of the following circumstances:

1. Not signing up for a grading slot before the sign-up period closes. Unless otherwise stated, the period closes at the same time as the submission deadline.
2. Not showing up at your grading slot.

If none of the provided slots work for you, or if you have already missed your slot, follow instructions outlined in the given Piazza post for F2F grading to get graded after the grading period ends. Also, please note the following:

1. You will need to explain to your TA why you are getting graded late, and may be asked to present documents to justify your hardship. The TA may remove the no-show penalty as long as he/she deems the justification to be reasonable.
2. In the past some students reported that their names disappeared mysteriously due to technical glitches, and the spreadsheet's edit history has no trace of it. So double check that your name is on the sign-up sheet after you sign up by refreshing the page.
3. Please do not try to modify other students' times or act in bad faith. Again, the sign-up sheet has history.

References

- [1] T. Liu, A. W. Bargteil, J. F. O'Brien, and L. Kavan. *Fast Simulation of Mass-Spring Systems*. ACM Transactions on Graphics (SIGGRAPH Asia), 32(6), 2013.
- [2] NVIDIA. Warp Documentation. <https://nvidia.github.io/warp/>